

Commons — Developer Guide

A plain-English overview of what Commons is, how the codebase is organized, and how everything fits together.

What is Commons?

Commons is a **neighborhood-scoped social planning app for women**. People post loose plans ("yoga Saturday", "coffee this week") and others can say they're interested or going. The product is **iOS-first** (built with Capacitor) and **invite-only** at launch.

Core ideas:

- **Loose by default** — plans don't need a locked time or address
- **Neighborhood-first** — the feed is scoped to where you live
- **Low friction** — phone sign-in, tap to join, minimal forms
- **Plans have group chat** — every plan gets a thread; DMs stay plan-scoped

The marketing site lives at `/welcome`. The real app lives behind sign-in.

Repo layout

```
Commons/
├─ client/           React app (Vite + TypeScript + Capacitor for iOS)
├─ server/           Express API (TypeScript)
├─ shared/           Older/shared type stubs (mostly superseded)
├─ commons-prd.md    Product spec (detailed feature reference)
├─ package.json      Root scripts: install, build, start (production)
├─ railway.json      Deploy config for Railway
└─ DEV.md            This file
```

Client (`client/`) — everything the user sees:

- `src/pages/` — full screens (Feed, PlanDetail, Chat, Onboarding, etc.)
- `src/components/` — reusable UI (PlanCard, BottomNav, PollCard, etc.)
- `src/context/` — React context (auth, theme)
- `src/api/` — HTTP client that talks to the backend
- `src/lib/` — helpers (platform detection, push, geolocation, image resize)
- `src/types/shared.ts` — TypeScript types shared with the server (keep in sync)
- `src/index.css` — most global styling
- `ios/` — native iOS shell (Xcode project, Capacitor)
- `public/` — static assets (landing photos, favicon, card image samples)

Server (`server/`) — API and business logic:

- `src/index.ts` — Express app entry, route mounting, production static serving
- `src/routes/` — API endpoints grouped by area
- `src/store.ts` — in-memory data + `data.json` persistence (the hot path for reads)
- `src/mongoMirror.ts` — writes through to MongoDB
- `src/hydrate.ts` — loads Mongo into memory on server start
- `src/userRepo.ts` — user read/write helpers (always go through `store`)

- `src/models/` — Mongoose schemas
- `src/lib/` — auth, notifications, SMS, recommendations, Google Cloud, etc.
- `src/seed.ts` — demo neighborhoods, users, and plans for local dev

Tech stack

Layer	Technology
Frontend	React 19, TypeScript, Vite, React Router
Mobile	Capacitor 7 (iOS)
Backend	Express 5, TypeScript
Database	MongoDB (via Mongoose) + in-memory snapshot
Auth	Phone + SMS code (Twilio Verify), JWT session
Hosting	Railway (single service serves API + built client)
Storage	Google Cloud Storage (optional, for event card images)
Maps	Google Places API (venue search, autocomplete)

Running locally

You need **Node 20+**.

1. Install dependencies

From the repo root:

```
npm run install:all
```

Or separately:

```
npm install --prefix client
npm install --prefix server
```

2. Configure the server

Copy the example env file and fill in what you need:

```
cp server/.env.example server/.env
```

Minimum for local dev:

- `PORT=4000`
- `APP_URL=http://localhost:5173`
- `SESSION_SECRET` and `MAGIC_LINK_SECRET` — any random strings
- `MONGODB_URI` — optional locally; without it, data lives in `server/data.json`

For real phone sign-in, add Twilio Verify credentials (see [Environment variables](#)).

3. Start the API

```
cd server && npm run dev
```

Runs on <http://localhost:4000> with hot reload (`tsx watch`).

4. Start the client

In a second terminal:

```
cd client && npm run dev
```

Runs on <http://localhost:5173>. In dev, the client calls the API at `http://localhost:4000` automatically (see `client/src/api/http.ts`).

5. Open the app

- **Web (marketing):** <http://localhost:5173/welcome>
- **Web (app, needs auth):** <http://localhost:5173/>
- **Onboarding:** <http://localhost:5173/onboarding>

Without Mongo, the server seeds demo data on first run (Philly neighborhoods, fake users, sample plans). Demo phones look like `+15555550100` – `+15555550111` .

Production build (local test)

```
npm run build    # builds client + server
npm start       # serves API + client/dist on PORT
```

Environment variables

All server config lives in `server/.env` . See `server/.env.example` for the full list.

Variable	Purpose
PORT	API port (default 4000)
MONGODB_URI	MongoDB connection string. Required in production.
APP_URL	Public URL of the web app (CORS, SMS links)
API_URL	Public URL of the API (some server-side link building)
SESSION_SECRET	Signs JWT session tokens
MAGIC_LINK_SECRET	Signs magic-link tokens

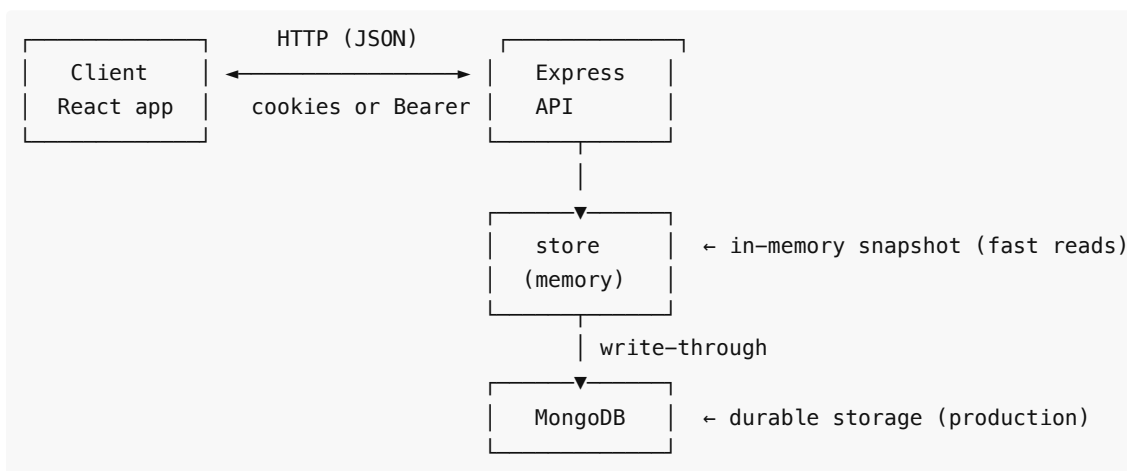
TWILIO_ACCOUNT_SID / TWILIO_AUTH_TOKEN / TWILIO_VERIFY_SERVICE_SID	Phone verification
TWILIO_FROM_NUMBER	Optional — SMS invites / nudges
GOOGLE_MAPS_API_KEY	Venue autocomplete and place details
GCS_BUCKET	Card image library in Google Cloud Storage
SEED_DEMO_ACCOUNTS	Set to 1 to allow demo seed in production (normally off)

Client env (for native builds, set before `vite build`):

Variable	Purpose
VITE_API_URL	Backend URL. Required for iOS/Android — the WebView can't use relative URLs.
VITE_PUBLIC_WEB_ORIGIN	Public web origin for share links (defaults to <code>https://commons.app</code>)

Architecture overview

The big picture



On startup:

1. Connect to Mongo (if `MONGODB_URI` is set)
2. **Hydrate** — pull all collections from Mongo into the in-memory `store`
3. **Seed** — if the database is empty, insert demo Philly data
4. Start background **nudge schedulers** (reminders, digests)

Every API read goes through the in-memory `store` (sync, fast). Every write updates `store` and **mirrors** to Mongo in the background. Locally without Mongo, writes also save to `server/data.json`.

Why this pattern?

The app was built for fast iteration. The in-memory layer keeps route handlers simple and synchronous. Mongo is the source of truth in production — if the server restarts, hydration reloads everything.

Important: In production, `data.json` is ephemeral. If Mongo mirroring fails, data can be lost. Check server logs for `[mongoMirror]` errors.

Authentication

How sign-in works

1. User enters phone number → `POST /api/auth/request-code`
2. Twilio Verify sends a 6-digit SMS (or a dev fallback if Twilio isn't configured)
3. User enters code → `POST /api/auth/verify-code`
4. Server creates/finds the user and issues a **JWT session token**

How sessions are stored

Platform	Method
Web browser	httpOnly cookie named <code>session</code>
iOS (Capacitor)	Bearer token in <code>Authorization</code> header, stored via Capacitor Preferences

The `requireAuth` middleware accepts either a cookie or a Bearer token.

Protected routes

`client/src/App.tsx` wraps most pages in a `<Protected>` guard:

- No user → redirect to `/onboarding` (native) or `/welcome` (web)
- User but onboarding incomplete → redirect to `/onboarding`
- Profile is reachable during onboarding (`allowIncomplete`)

Onboarding flow

`client/src/pages/Onboarding.tsx` walks new users through:

1. Phone number
2. SMS verification code
3. Exclusive invite code gate (temporary launch mechanic — code `commonsphl`)
4. Age confirmation
5. Neighborhood picker
6. Interest selection (up to 3)
7. Avatar (photo or DiceBear-style builder)
8. Legal consent (terms, privacy, community guidelines)

When complete, `onboardingComplete` is set on the user and they land on the feed.

Admin access

Certain phone numbers are hardcoded as admins (`server/src/lib/adminPhones.ts`). Admins can open `/admin` for user lookup and card-image management.

Main screens and routes

Route	Page	What it does
-------	------	--------------

/welcome	Landing	Public marketing page + waitlist (Google Form modal)
/onboarding	Onboarding	Phone sign-in and profile setup
/	Feed	Home — week-at-a-glance plan cards, filters, pull-to-refresh
/explore	Explore	Discover plans (near me, search)
/plans/new	CreatePlan	Post a new plan
/plans/:id	PlanDetail	Plan info, RSVP, invite, share, get-there
/plans/:id/edit	EditPlan	Host edits a plan
/plans/:planId/chat	Chat	Group chat for a plan (messages, polls, reactions)
/messages	Messages	Inbox of all plan conversations
/my-plans	MyPlans	Plans you host, joined, or saved
/profile/:userId	Profile	User profile, network, interests
/network	Network	Your connections
/invite	Invite	Share invite codes
/settings	Settings	Account settings menu
/settings/interests	SettingsInterests	Edit interests
/settings/notifications	NotificationPrefs	Notification toggles
/notifications	Notifications	In-app notification list
/legal/:slug	Legal	Terms, privacy, guidelines
/admin	Admin	Internal admin tools

Bottom nav (when visible): Home · Explore · Make a Plan (+) · Chats · Profile

Core product flows

Posting a plan

1. User taps **Make a Plan** → `CreatePlanPage`
2. Fills title, date/time (can be flexible), location (Google Places or neighborhood), tags, visibility
3. `POST /api/plans` creates the plan and a group conversation
4. Plan appears in the feed for users in the same neighborhood scope

Plan kinds:

- `standard` — a normal hosted plan
- `looking_for` — a casual “anyone want to do X?” post that can be **locked in** later with a real time/place

Visibility:

- `everyone` — anyone in the neighborhood scope
- `network` — only your connections (+ people already RSVP'd)
- `community` — filtered by a specific interest tag

Joining a plan

On plan detail, users tap **I'm in** or **Interested**:

- `PUT /api/plans/:id/participation` with state `going` or `interested`
- Host-only plans may require approval (`joinType: "approval"`) via `POST /api/plans/:id/approve`

Group chat

Every plan has one conversation, created automatically.

- List all chats: `GET /api/conversations`
- Open plan chat: `GET /api/plans/:planId/conversation`
- Send message: `POST /api/conversations/:id/messages`
- Polls, reactions, and leave-chat are also on the chat router

Chat is only for people on the plan — you can't cold-DM someone.

Network (connections)

After sharing a plan, users may get a **network prompt** ("add Jamie to your network?").

- `GET /api/auth/network-prompt` — pending prompt
- `POST /api/auth/network-add / network-accept / network-decline`
- `GET /api/auth/network` — list connections

Network affects plan visibility and profile social-link visibility.

Feed ranking

`GET /api/plans` returns plans ranked by `server/src/lib/recommend.ts` :

- Interest tag overlap (highest weight)
- Neighborhood proximity (same hood > adjacent hood)
- Social proof (how many people are going)
- Host quality, recency, prior declines

Plans are filtered to the viewer's neighborhood scope and visibility rules before ranking.

Notifications

In-app notifications are created by `server/src/lib/notify.ts` → `emit()` . Each kind respects user prefs and deduplication.

Kinds include: someone joined your plan, plan tomorrow, plan in 2 hours, new chat message, plan cancelled, weekly digest, network requests, and more.

Push notifications (iOS only): `client/src/lib/push.ts` registers the device token via `POST /api/devices/register` . Delivery wiring lives server-side when configured.

Background nudges

`server/src/lib/nudges.ts` runs on timers after server start:

- Plan reminders (tomorrow, 2 hours before)
- Weekend digest SMS (Thu/Fri)
- Venue return nudges ("new plan at a place you've been")
- Lock-in texts for interested users when a flexible plan gets confirmed

API reference (summary)

All routes are under `/api`. Most require auth (`requireAuth` middleware).

Auth — `/api/auth`

Method	Path	Description
POST	<code>/request-code</code>	Send SMS verification code
POST	<code>/verify-code</code>	Verify code, create session
GET	<code>/me</code>	Current user profile
PATCH	<code>/me</code>	Update profile / onboarding fields
POST	<code>/logout</code>	Clear session cookie
DELETE	<code>/me</code>	Delete account
GET	<code>/invite-codes</code>	User's invite codes
POST	<code>/redeem-code</code>	Redeem someone else's code
GET/POST	<code>/network-*</code>	Network prompts, add, accept, decline
POST	<code>/save-plan</code>	Pin/save a plan

Plans — `/api/plans`

Method	Path	Description
GET	<code>/preview</code>	Public plan preview (no auth, for onboarding tease)
GET	<code>/</code>	Feed plans (ranked)
POST	<code>/</code>	Create plan
GET	<code>/plans/:id</code>	Plan detail
PATCH	<code>/plans/:id</code>	Edit plan (host)
PUT	<code>/plans/:id/participation</code>	RSVP (going / interested)
DELETE	<code>/plans/:id/participation</code>	Leave / remove RSVP
POST	<code>/plans/:id/lock</code>	Lock in a flexible plan

POST	/plans/:id/cancel	Cancel plan
POST	/plans/:id/invite	Invite specific users
POST	/plans/:id/propose-time	Propose a time change
POST	/plans/:id/apply-time	Apply proposed time
...	host transfer, up-for-grabs, claim-host, suggestions	

Chat — /api/...

Mounted at /api with its own path prefixes:

- GET /conversations — inbox
- GET /plans/:planId/conversation — plan's chat
- GET/POST /conversations/:id/messages — read/send messages
- Poll and reaction endpoints under the same conversation paths

Other routes

Prefix	Purpose
/api/profile/:userId	Public profile view
/api/neighborhoods	List neighborhoods
/api/places/*	Google Places proxy (autocomplete, details, nearby)
/api/venues	Plans grouped by venue
/api/notifications	In-app notifications
/api/devices	Push token registration
/api/feedback	Post-plan feedback
/api/card-images	Event card image library
/api/link-preview	OG preview for shared links
/api/admin/*	Admin tools (card images, user lookup)
/api/health	Health check

Data model (main entities)

All records live in `server/src/store.ts` as TypeScript interfaces. Mongoose models in `server/src/models/` mirror them for Mongo.

User

Phone number, name, neighborhoods, interests, avatar, onboarding status, network connections, saved plans, notification prefs, legal consent timestamps.

Neighborhood

Named areas (e.g. Rittenhouse, Fishtown) with adjacency for proximity scoring. Seeded for Philadelphia at launch.

Plan

Title, host, location, date/time, tags, visibility, capacity, join type, plan kind (`standard` / `looking_for`), lock/cancel state, optional flyer image and link.

Participation

Links a user to a plan with state: `going` or `interested` .

Conversation + Message

One conversation per plan. Messages can be text, polls (with votes), or system events. Reactions supported.

Notification

In-app alert with kind, body, dedup key, and optional links to plan/chat/profile.

InviteCode

Launch invite codes — each user gets a fixed number to share.

Shared types

API request/response shapes are defined in:

- `client/src/types/shared.ts`
- `server/src/types/shared.ts`

Keep both files in sync when changing types. They include interests, plan DTOs, user DTOs, notification prefs, chat types, etc.

There is also an older `shared/types.ts` at the repo root — prefer the copies inside client and server.

iOS / Capacitor

The React app is wrapped in a native iOS shell via Capacitor.

File	Purpose
<code>client/capacitor.config.ts</code>	App ID (<code>com.oncommons.mvp</code>), web dir
<code>client/ios/</code>	Xcode project

Building for iOS

1. Set `VITE_API_URL` to your production API URL
2. Build the web app: `cd client && npm run build`
3. Sync to iOS: `npx cap sync ios`
4. Open in Xcode: `npx cap open ios`

Native capabilities used:

- **Push notifications** — @capacitor/push-notifications
- **Camera / photo picker** — @capacitor/camera
- **Geolocation** — @capacitor/geolocation
- **Clipboard** — share sheets
- **Preferences** — auth token storage

client/src/lib/platform.ts exposes isNative() — used throughout to branch web vs native behavior (auth, API base URL, push, etc.).

Deployment (Railway)

railway.json configures:

- **Build:** npm run build (installs client + server deps, builds both)
- **Start:** npm start → node server/dist/index.js

In production (NODE_ENV=production):

- Express serves client/dist as static files
- All non-API routes fall through to index.html (SPA)
- API stays at /api/*

Set all server/.env variables in Railway's environment. **MongoDB URI is required.**

Admin tools

Route: /admin (requires admin phone number)

Capabilities:

- User lookup and summary stats
 - Card image library management (upload, seed defaults, delete)
 - Uses Google Cloud Storage when GCS_BUCKET is configured; otherwise falls back to inline data URLs
-

Seeding and test data

Automatic seed

server/src/seed.ts runs on startup if the database is empty. It creates:

- 10 Philly neighborhoods (with stable UUIDs — don't change these in prod)
- ~50 demo users (+15555550100 and up)
- ~50 sample plans with participations and chat messages

Disabled in production unless SEED_DEMO_ACCOUNTS=1 .

Manual mock seed

```
cd server && npm run seed:mock
```

Other scripts

- `server/src/scripts/redateMockPlans.ts` — shift plan dates forward
 - `server/src/scripts/seedPastTogether.ts` — seed shared-plan history for network testing
 - `server/src/scripts/diagnoseMockDates.ts` — debug plan date issues
-

Styling and design

- Global CSS in `client/src/index.css` — most component styling uses CSS classes, not CSS-in-JS
 - Brand colors: navy `#141130`, red `#BF2B2B`, beige `#EBE4DA`
 - Fonts: Plus Jakarta Sans (display), Poppins (body)
 - Landing page (`Landing.tsx`) uses inline styles for the marketing layout
 - App screens use the shared design system in `index.css`
-

External services

Service	Used for
Twilio Verify	Phone sign-in SMS codes
Twilio Messaging	Transactional SMS (invites, nudges) — optional
MongoDB Atlas	Production database
Google Places	Venue search and autocomplete
Google Cloud Storage	Admin card image library
Google Forms	Waitlist on landing page (embedded iframe)
DiceBear	Generated avatar URLs when no photo
Railway	Hosting

Common dev tasks

Add a new API endpoint

1. Create or extend a router in `server/src/routes/`
2. Mount it in `server/src/index.ts`
3. Add store methods in `server/src/store.ts` if new data is needed
4. Add mongo mirror calls if it's a new collection
5. Call it from the client via `api()` in `client/src/api/http.ts`

Add a new page

1. Create `client/src/pages/YourPage.tsx`
2. Add a route in `client/src/App.tsx`
3. Add nav link in `BottomNav.tsx` or `TopBar.tsx` if needed

Change a shared type

Edit both `client/src/types/shared.ts` and `server/src/types/shared.ts` .

Test phone sign-in locally

Without Twilio, the server may accept a dev fallback code (check `server/src/lib/verify.ts`). With Twilio configured, use a real phone number.

Where to read more

File	Contents
<code>commons-prd.md</code>	Full product spec — features, UX rationale, future roadmap
<code>server/.env.example</code>	All environment variables with comments
<code>server/src/store.ts</code>	Every data record shape and store methods
<code>client/src/App.tsx</code>	All routes and auth guards
<code>server/src/index.ts</code>	Server bootstrap and route list

Quick troubleshooting

Problem	Likely cause
401 on every API call	Not signed in, or session expired. Try onboarding again.
Empty feed	User has no neighborhood set, or no plans in scope. Check seed ran.
iOS can't reach API	VITE_API_URL not set at build time
Data disappears on deploy	Mongo not connected; check MONGODB_URI and [mongoMirror] logs
Places autocomplete empty	GOOGLE_MAPS_API_KEY missing or restricted
SMS not sending	Twilio credentials missing or Verify service not set up

Last updated: July 2026